COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 / Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	August 18, 2026	**DATE SUBMITTED:**	September 1, 2026
NAME:	Reyes, Fenzie Aaron A.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 6.1 SEARCHING TECHNIQUES

1. PROCEDURE OUTPUT

What procedures did you perform?

CODE INPUT (UNDER MAIN.CPP):

main.cpp | node.h | searchingAlgo.h

main.cpp > main()

```cpp
#include <iostream>
#include <cstdlib> // for generating random integers
#include <time.h> // will be used for our seeding function
#include "node.h"
#include "searchingAlgo.h"

const int max_size = 50;

int main(){

    // Generate a random number between 1 and 50
    int dataset[max_size];
    srand(time(0)); //seed the random number generator
    for(int i = 0; i < max_size; i++){
        dataset[i] = rand(); //random number between 1 and 50
    }
    for(int i = 0; i < max_size; i++){
        std::cout << dataset[i] << " ";
    }
    std::cout << std::endl;

    int userInput;
    std::cout << "What number to search?";
    std::cin >> userInput;
    std::cout << "========================================" << std::endl;
    std::cout << "Array result LS: ";
    linearSearchArr(dataset, max_size, userInput); //search for the entered number in the dataset
    std::cout << "Array result BS: ";
    binarySearchArr(dataset, max_size, userInput); //search for the entered number in the dataset
    std::cout << "----------------------------------------" << std::endl;

    // Create a linked list
    Node<char>* name1 = newNode('F');
    Node<char>* name2 = newNode('E');
    Node<char>* name3 = newNode('N');
    Node<char>* name4 = newNode('Z');
```

```
    Node<char>* name5 = newNode('I');
    Node<char>* name6 = newNode('E');

    // Create the Name
    name1->next = name2;
    name2->next = name3;
    name3->next = name4;
    name4->next = name5;
    name5->next = name6;

    // Demonstration
    std::cout << "========================================" << std::endl;
    std::cout << "Linked List result LS: ";
    linearSearchLL(name1, 'R');
    linearSearchLL(name1, 'r');
    std::cout << "Linked List result LS: ";
    binarySearchLL(name1, 'R');
    binarySearchLL(name1, 'r');
    std::cout << "----------------------------------------" << std::endl;

    return 0;
}
```

CODE INPUT UNDER (NODE.H)

main.cpp | node.h | searchingAlgo.h

node.h > newNode<T>(T)

```
#ifndef NODE_H
#define NODE_H

template <typename T>
class Node{
public:
    T data;
    Node<T>* next;
};

template <typename T>
Node<T>* newNode(T newData){
    Node<T>* newNode = new Node<T>();
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}



#endif
```

CODE INPUT UNDER (SEARCHINGALGO.H)

main.cpp | node.h | searchingAlgo.h

searchingAlgo.h > binarySearchArr<T>(T [], int, T)

```cpp
#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

#include <iostream>
#include <algorithm> // allows sortings

template <typename T>
void linearSearchArr (T list[], int arrSize, T dataFind){
    // Initialize i = 0
    int i = 0;
    // While i <= n
    while (i < arrSize) {
        //if (item = data[i])
        if (list[i] == dataFind) {
            // Print Searching is Successful
            std::cout << "Searching is Successful" << std::endl;
            std::cout << "Element found at index " << i << std::endl;
            return;
        }
        i++;
    }
    // Print Searching is Unsuccessful
    std::cout << "Searching is Unsuccessful" << std::endl;
}

template <typename T>
void linearSearchLL(Node<T>* head, T dataFind){

    int i = 0;
    // While i != nullptr
    while(head != nullptr){
        //
        if(head->data == dataFind){
            std::cout << "Searching is Successful" << std::endl;
            return;
        }
        head = head->next;
    }
    std::cout << "Searching is Unsuccessful" << std::endl;
}

template <typename T>
Node<T>* getMiddle(Node<T>* start, Node<T>* last){

    Node<T>* slow = start;
    Node<T>* fast = start;

    while(fast != last && fast->next != last){

        // move slow one step
        slow = slow->next;

        // move fast two steps
        fast = fast->next->next;
    }

    return slow;
}

// Linked List implementation Binary Search
template <typename T>
void binarySearchLL(Node<T>* start, T dataFind) {
    // Step 1: Initialize last = nullptr
    Node<T>* last = nullptr;

    // Step 2: Repeat while start != last
    while (start != last) {
        // mid = getMiddle(start, last)
```

```
        Node<T>* mid = getMiddle(start, last);

        if (mid == nullptr) {
            std::cout << "Searching is Unsuccessful" << std::endl;
            return;
        }

        // if (dataFind == mid->data)
        if (mid->data == dataFind) {
            std::cout << "Searching is Successful" << std::endl;
            return;
        }
        // else if (dataFind < mid->data) then last = mid
        else if (dataFind < mid->data) {
            last = mid;
        }
        // else start = mid->next
        else {
            start = mid->next;
        }
    }

    // Step 3: Print Searching is Unsuccessful
    std::cout << "Searching is Unsuccessful" << std::endl;
}

// Array implementation Binary Search
template <typename T>
void binarySearchArr(T list[], int arrSize, T dataFind) {
    // IMPORTANT: Sort array before binary search
    std::sort(list, list + arrSize);

    // Step 1: low = 0, up = n-1
    int low = 0;
    int up = arrSize - 1;

    // Step 2: Repeat while low <= up
    while (low <= up) {
        // mid = (low + up) / 2
        int mid = (low + up) / 2;

        // if (dataFind == list[mid])
        if (list[mid] == dataFind) {
            std::cout << "Searching is Successful" << std::endl;
            std::cout << "Element found at index " << mid << std::endl;
            return;
        }
        // else if (dataFind < list[mid]) then up = mid - 1
        else if (dataFind < list[mid]){
            up = mid - 1;
        }
        // else low = mid + 1
        else {
            low = mid + 1;
        }
    }

    // Step 3: Print Searching is Unsuccessful
    std::cout << "Searching is Unsuccessful" << std::endl;
}

#endif
```

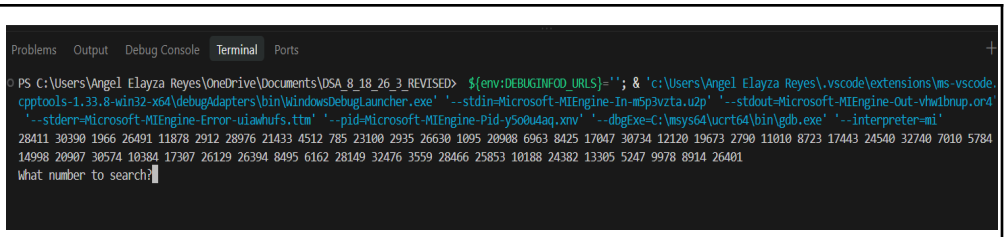
Screenshot	
Observation	The program successfully generated an array containing 50 random integers. The values are initially displayed in their generated order.

Table 6-1. Data Generated and Observations.

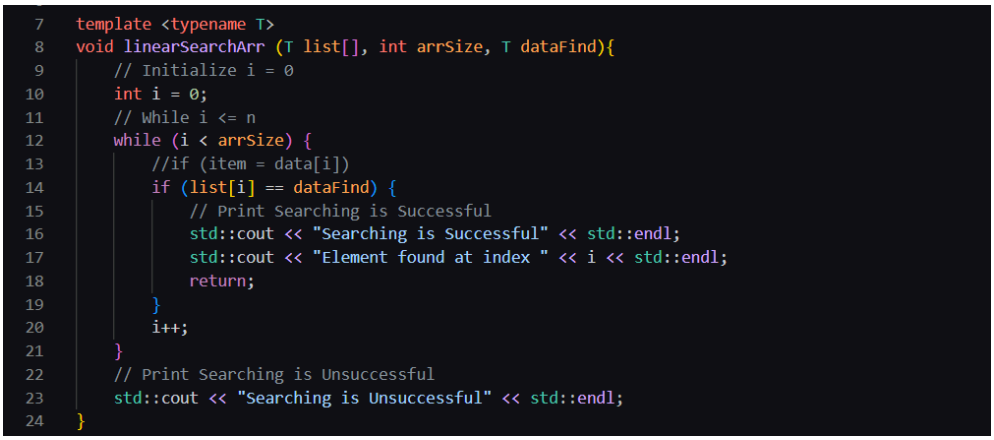
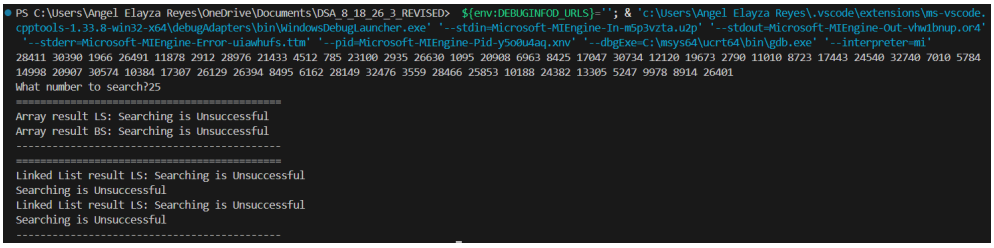
Screenshot	 
Observation	The linear search checks the elements sequentially from the beginning of the array until the target value is found. The search is successful when the target value matches an element in the array.

Table 6-2a. Linear Search for Arrays

Screenshot

```cpp
template <typename T>
void linearSearchLL(Node<T>* head, T dataFind){

    int i = 0;
    // While i != nullptr
    while(head != nullptr){
        //
        if(head->data == dataFind){
            std::cout << "Searching is Successful" << std::endl;
            return;
        }
        head = head->next;
    }
    std::cout << "Searching is Unsuccessful" << std::endl;
}
```

```
==========================================
Linked List result LS: Searching is Unsuccessful
Searching is Unsuccessful
Linked List result LS: Searching is Unsuccessful
Searching is Unsuccessful
------------------------------------------
```

Observation

The linear search traverses the linked list node by node from head until the target is found or the end of the list has been reached. The search made by me is 'R', and is not successful since neither character exists in the linked list.

Table 6-2b. Linear Search for Linked List

Screenshot

```cpp
// Array implementation Binary Search
template <typename T>
void binarySearchArr(T list[], int arrSize, T dataFind) {
    // IMPORTANT: Sort array before binary search
    std::sort(list, list + arrSize);
    // Step 1: low = 0, up = n-1
    int low = 0;
    int up = arrSize - 1;
    // Step 2: Repeat while low <= up
    while (low <= up) {
        // mid = (low + up) / 2
        int mid = (low + up) / 2;

        // if (dataFind == list[mid])
        if (list[mid] == dataFind) {
            std::cout << "Searching is Successful" << std::endl;
            std::cout << "Element found at index " << mid << std::endl;
            return;
        }
        // else if (dataFind < list[mid]) then up = mid - 1
        else if (dataFind < list[mid]){
            up = mid - 1;
        }
        // else low = mid + 1
        else {
            low = mid + 1;
        }
    }
    // Step 3: Print Searching is Unsuccessful
    std::cout << "Searching is Unsuccessful" << std::endl;
}
```

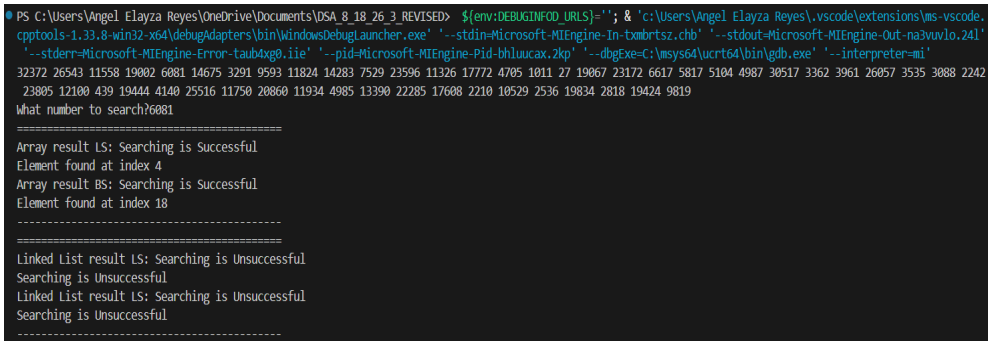
	
Observation	Binary search first sorts the array and then repeatedly divides the search range in half. It is successful when the target is found in the sorted array. The index may differ from the linear search because the array is sorted before the binary search.

Table 6-3a. Binary Search for Arrays

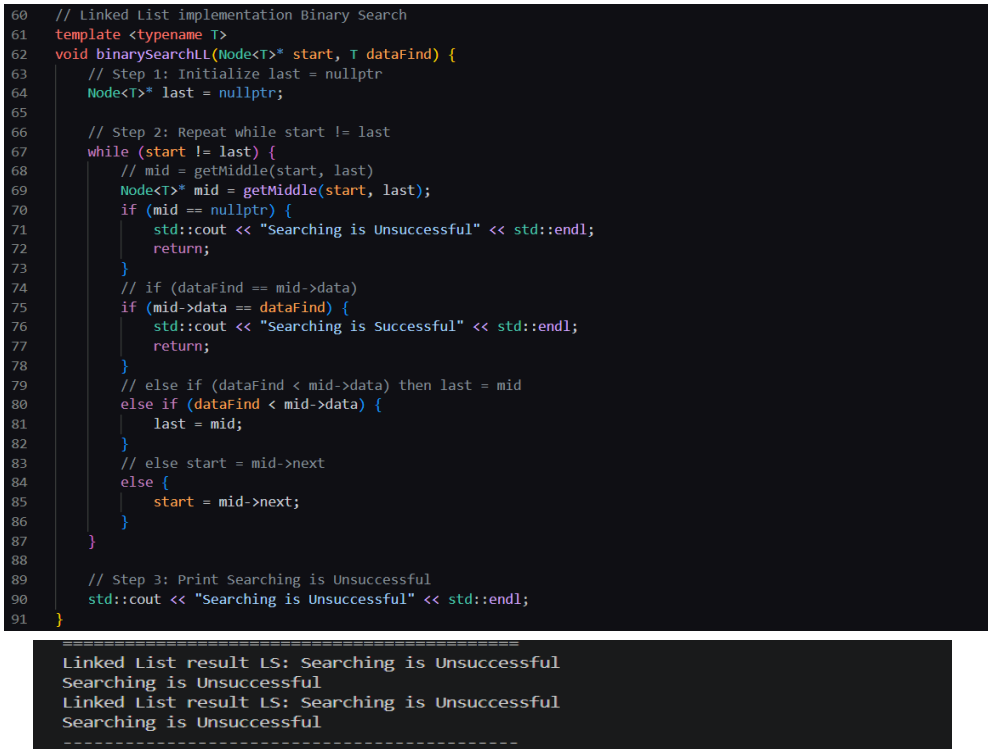
Screenshot	
Observation	The binary search for the linked list attempts to locate the middle node and reduce the search range. However, binary search requires the data to be sorted. Since the linked list contains the characters F, E, N, Z, I, E in an unsorted order, the binary search is not reliable for this particular list.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILOB UNDER (SEARCHINGALGO.H)

main(1).cpp | searchingAlgo(1).h

searchingAlgo(1).h > ...

```cpp
#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

#include <iostream>

// node used for the linked list
template <typename T>
class Node {
public:
    T data;
    Node<T>* next;

    Node(T value) {
        data = value;
        next = nullptr;
    }
};

// ========================================
// Problem 1 - Sequential Search using Array
// ========================================

template <typename T>
int sequentialSearchArray(T arr[], int size, T key) {
    int comparisons = 0;

    // check each element starting from the first one
    for (int i = 0; i < size; i++) {
        comparisons++;
        // stop once the key is found
        if (arr[i] == key) {
            return comparisons;
        }
    }
    return comparisons;
}
```

```cpp
// ========================================
// Problem 1 - Sequential Search using Linked List
// ========================================
template <typename T>
int sequentialSearchLinkedList(Node<T>* head, T key) {
    int comparisons = 0;
    // go through the nodes one at a time
    while (head != nullptr) {
        comparisons++;
        // check if the current node contains the key
        if (head->data == key) {
            return comparisons;
        }
        // move to the next node
        head = head->next;
    }
    return comparisons;
}
```

```cpp
// ========================================
// Problem 2 - Count Repeating Instances
// ========================================
template <typename T>
int countOccurrences(T arr[], int size, T key) {
    int count = 0;
    // check the whole list so every occurrence is counted
    for (int i = 0; i < size; i++) {
        if (arr[i] == key) {
            count++;
        }
    }
    return count;
}
```

```cpp
// Problem 3 - Iterative Binary Search
// ========================================
template <typename T>
int binarySearch(T arr[], int size, T key) {
    int low = 0;
    int high = size - 1;
    int iteration = 1;
    // continue searching while the range is valid
    while (low <= high) {
        // find the middle position
        int mid = (low + high) / 2;
        std::cout << "Iteration " << iteration
                  << ": Low = " << low
                  << ", High = " << high
                  << ", Mid = " << mid
                  << ", Value = " << arr[mid]
                  << std::endl;

        // check if the middle value is the key
        if (arr[mid] == key) {
            return mid;
        }
        // move to the left half if the key is smaller
        else if (key < arr[mid]) {
            high = mid - 1;
        }
        // otherwise move to the right half
        else {
            low = mid + 1;
        }
        iteration++;
    }

    return -1;
}
```

```cpp
// ========================================
// Problem 4 - Recursive Binary Search
// ========================================
template <typename T>
int recursiveBinarySearch(T arr[], int low, int high, T key, int& iteration) {
    // stop when there is no range left to search
    if (low > high) {
        return -1;
    }
    // find the middle position of the current range
    int mid = (low + high) / 2;
    std::cout << "Iteration " << iteration
              << ": Low = " << low
              << ", High = " << high
              << ", Mid = " << mid
              << ", Value = " << arr[mid]
              << std::endl;
    // return the position when the key is found
    if (arr[mid] == key) {
        return mid;
    }
    iteration++;
    // search the left half if the key is smaller
    if (key < arr[mid]) {

        return recursiveBinarySearch(arr, low, mid - 1, key, iteration);
    }

    // otherwise search the right half
    return recursiveBinarySearch(arr, mid + 1, high, key, iteration);
}

#endif
```

ILOB UNDER (MAIN.CPP)

main(1).cpp searchingAlgo(1).h

main(1).cpp > main()

```cpp
#include <iostream>
#include "searchingAlgo(1).h"

int main() {

    // ================================================
    // Problem 1 - Sequential Search
    // ================================================

    int list[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};

    int size = 10;
    int key = 18;

    std::cout << "=========================================\n";
    std::cout << "Problem 1 - Sequential Search\n";
    std::cout << "=========================================\n";

    std::cout << "List: ";

    // display the given list
    for (int i = 0; i < size; i++) {
        std::cout << list[i] << " ";
    }

    std::cout << "\nKey: " << key << "\n\n";


    // search for the key using an array
    int arrayComparisons =
        sequentialSearchArray(list, size, key);

    // create a linked list using the same values
    Node<int>* head = new Node<int>(list[0]);
    Node<int>* current = head;
```

main(1).cpp searchingAlgo(1).h

main(1).cpp > main()

```cpp
    // connect each node to the next one
    for (int i = 1; i < size; i++) {
        current->next = new Node<int>(list[i]);
        current = current->next;
    }

    // search for the key using the linked list
    int linkedListComparisons =
        sequentialSearchLinkedList(head, key);

    std::cout << "Array comparisons: "
              << arrayComparisons << std::endl;

    std::cout << "Linked List comparisons: "
              << linkedListComparisons << std::endl;
```

```cpp
    // ================================================
    // Problem 2 - Count Repeating Instances
    // ================================================
    std::cout << "\n=========================================\n";
    std::cout << "Problem 2 - Count Repeating Instances\n";
    std::cout << "=========================================\n";
    std::cout << "List: ";

    // display the list
    for (int i = 0; i < size; i++) {
        std::cout << list[i] << " ";
    }

    std::cout << "\nSearch element: " << key << std::endl;

    // count every time the key appears
    int occurrences =
        countOccurrences(list, size, key);

    std::cout << "Number of occurrences of "
              << key << ": "
              << occurrences << std::endl;
```

```cpp
    // ================================================
    // Problem 3 - Iterative Binary Search
    // ================================================
    std::cout << "\n=========================================\n";
    std::cout << "Problem 3 - Iterative Binary Search\n";
    std::cout << "=========================================\n";

    // binary search needs a sorted list
    int sortedList[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
    int sortedSize = 10;
    int binaryKey = 8;

    std::cout << "List: ";

    for (int i = 0; i < sortedSize; i++) {
        std::cout << sortedList[i] << " ";
    }

    std::cout << "\nSearching for: "
          << binaryKey << "\n\n";

    // perform the iterative binary search
    int result =
        binarySearch(sortedList, sortedSize,binaryKey);


    if (result != -1) {
        std::cout << "\nSearching is Successful\n";
        std::cout << "Element found at index "
                  << result << std::endl;
    }else {
        std::cout << "\nSearching is Unsuccessful\n";
    }
```

```cpp
    // ================================================
    // Problem 4 - Recursive Binary Search
    // ================================================
    std::cout << "\n========================================\n";
    std::cout << "Problem 4 - Recursive Binary Search\n";
    std::cout << "========================================\n";

    std::cout << "List: ";

    for (int i = 0; i < sortedSize; i++) {
        std::cout << sortedList[i] << " ";
    }

    std::cout << "\nSearching for: "
              << binaryKey << "\n\n";

    // start the recursive search at the first and last index
    int iteration = 1;

    int recursiveResult =
        recursiveBinarySearch(sortedList, 0, sortedSize - 1, binaryKey, iteration);

    if (recursiveResult != -1) {
        std::cout << "\nSearching is Successful\n";
        std::cout << "Element found at index "
                  << recursiveResult << std::endl;
    }else {
        std::cout << "\nSearching is Unsuccessful\n";
    }

    return 0;
}
```

CODE OUTPUT ON TERMINAL

```
========================================
Problem 1 - Sequential Search
========================================
List: 15 18 2 19 18 0 8 14 19 14
Key: 18

Array comparisons: 2
Linked List comparisons: 2

========================================
Problem 2 - Count Repeating Instances
========================================
List: 15 18 2 19 18 0 8 14 19 14
Search element: 18
Number of occurrences of 18: 2

========================================
Problem 3 - Iterative Binary Search
========================================
List: 3 5 6 8 11 12 14 15 17 18
Searching for: 8

Iteration 1: Low = 0, High = 9, Mid = 4, Value = 11
Iteration 2: Low = 0, High = 3, Mid = 1, Value = 5
Iteration 3: Low = 2, High = 3, Mid = 2, Value = 6
Iteration 4: Low = 3, High = 3, Mid = 3, Value = 8

Searching is Successful
Element found at index 3
```

```
========================================
Problem 4 - Recursive Binary Search
========================================
List: 3 5 6 8 11 12 14 15 17 18
Searching for: 8

Iteration 1: Low = 0, High = 9, Mid = 4, Value = 11
Iteration 2: Low = 0, High = 3, Mid = 1, Value = 5
Iteration 3: Low = 2, High = 3, Mid = 2, Value = 6
Iteration 4: Low = 3, High = 3, Mid = 3, Value = 8

Searching is Successful
Element found at index 3
```

OBSERVATIONS:

1. **Problem 1 – Sequential Search:**
   - The sequential search checks the elements one by one starting from the first element. Since the key 18 is the second element in the list, only 2 comparisons are needed to find it. The result is the same for both the array and linked list approaches.

2. **Problem 2 – Count Repeating Instances**
   - The modified sequential search continues checking the entire list instead of stopping after finding the first occurrence. The key 18 appears twice in the given list, thus, the algorithm returns 2 occurrences.Since every element must be checked, a total of 10 comparisons has been done.

3. **Problem 3 – Iterative Binary Search**
   - The binary search starts by checking the middle element of the sorted list. Since the middle value is 11 and the key 8 is smaller, the search continues on the left side. It then checks 5, followed by 6, then it finally finds 8 at index 3. The key is found after 4 iterations.

4. **Problem 4 – Recursive Binary Search**
   - It follows the same search path as iterative binary search, but the key difference is that instead of using a loop, the function calls itself with a smaller search range. It checks the numbers 11, 5, 6 and 8. Then, the key is successfully found at index 3 after 4 iterations.

CONCLUSION AND LEARNINGS

Given all these activities, I learned how different searching techniques work and how they can be implemented in my VSCode. For instance, sequential search checks the elements one by one given by its name already, and it can also be used with both arrays and linked lists. On the other hand, binary search works by repeatedly dividing a sorted list into smaller parts. Furthermore, I also learned how to count repeated elements and how to implement binary search both iteratively and recursively. Understanding these topics was challenging at first because it took me some time before I actually understood how each part of the code works, but through watching videos online and visiting some websites which can help me, it paved the way for me to comprehend the topic well. Overall, all of these are crucial because searching is a common operation in programming, and choosing the right implementation can help in making your code efficient and effective. Additionally, understanding the topic will connect and help me in dealing with large amounts of data.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Grammarly and Copilot

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar correction and code assistance.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: The AI only paved the way for me to help debug the errors encountered, and in grammar checking. Other than that, it did not replace my desired outcome for the code.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.